

Resolução de Labirinto

relatório de projeto
AEDs 2

EXX
0 0 0
XXS

1. Introdução

O algoritmo tem como objetivo, dada uma matriz que representa um labirinto simples composto por caminhos livres e paredes, determinar se existe um percurso possível entre um ponto de entrada e um ponto de saída e exibir esse percurso.

No projeto proposto, o labirinto é fornecido por meio de um arquivo de texto contendo uma matriz de tamanho fixo 10x10, cada célula da matriz pode assumir um dos seguintes valores: '0' (caminho livre), 'E' (entrada do labirinto) ou 'S' (saída do labirinto).

O objetivo do algoritmo é ler essa matriz a partir do arquivo de texto, localizar a posição de entrada e, a partir dela, executar um processo de busca capaz de encontrar um caminho até a saída.

O problema pode ser dividido em três etapas principais:

- (i) a leitura e o armazenamento do labirinto em uma estrutura de matriz;
- (ii) a aplicação de um algoritmo de busca capaz de percorrer a matriz;
- (iii) a apresentação do resultado e trajeto realizado.

2. Estruturas de Dados

2.1 Matriz do labirinto

O labirinto é armazenado como uma matriz de caracteres de dimensões fixas, lide `char maze[10][10]`, sendo possível o acesso direto a qualquer célula meio dos índices de linha e coluna,.

Além disso, a própria matriz é reutilizada como mecanismo de marcação de espaços visitadas:, durante a busca cada posição visitada é sobrescrita com o valor '1', impedindo que o algoritmo entre em laço infinito revisitando a mesma célula.

2.2 Estrutura de registro

Para representar uma posição dentro da matriz, foi definida a estrutura “Pos”, contendo dois campos inteiros, `h` (linha) e `l` (coluna), utilizada para compor o vetor global `route[100]`, que funciona como um registro do caminho bem sucedido percorrido pelo algoritmo. Ao final da execução, o vetor `route` contém o caminho da saída até a entrada, sendo percorrido em ordem inversa para exibir o trajeto da entrada até a saída.

3. Algoritmos

3.1 Leitura do labirinto

A leitura do arquivo é realizada pela função **leMatriz**, que percorre a matriz 10x10 célula a célula utilizando dois laços.

Durante a leitura, a função busca também o caractere 'E', indicativo da entrada, armazenando sua posição nos parâmetros de saída **linhaEntrada** e **colunaEntrada**. A função **abreMatriz** é responsável por abrir o arquivo, invocar leMatriz e garantir o fechamento do arquivo ao final.

3.2 Busca do caminho

O algoritmo principal é a função recursiva **progredir**, que implementa uma busca em profundidade com retrocesso (backtracking) na matriz lida.

A cada chamada, a função compara a célula atual, se for a saída 'S', o caminho foi encontrado e a posição é empilhada no vetor '**route**', caso a célula seja um caminho livre 'O' ou a própria entrada 'E', a célula é marcada como visitada (recebendo '1') e a função tenta recursivamente se mover nas quatro direções.

Se alguma dessas chamadas recursivas retornar sucesso, a posição atual também é empilhada em '**route**' e a função retorna sucesso, propagando o resultado para os níveis anteriores da recursão em "efeito dominó".

Se nenhuma das quatro direções levar a uma solução, a função retorna falha, e a busca retrocede para tentar outra direção na célula anterior.

3.3 Análise de complexidade

Considerando uma matriz de $N \times N$ células (no caso do projeto, $N = 10$), a leitura do arquivo possui complexidade $O(N^2)$, já que cada célula é lida exatamente uma vez.

Já a função progredir apresenta, no pior caso, complexidade de tempo $O(N^2)$, como cada célula é marcada com '1' assim que é visitada pela primeira vez, ela nunca é visitada novamente pelas chamadas recursivas seguintes, de modo que a função realize no máximo quatro tentativas de leitura para cada uma das N^2 células.

É importante observar que embora o algoritmo garanta encontrar um caminho caso ele exista, ele não garante o caminho mais curto.

4. Makefile e Compilação

4.1 Estrutura do Makefile

A variável CC define o compilador utilizado (gcc);

CFLAGS define as flags de compilação, incluindo os avisos -Wall e -Wextra, que ajudam a identificar possíveis erros e más práticas no código, além do padrão da linguagem (-std=c11);

TARGET define o nome do executável gerado (maze);

SRC define o arquivo-fonte a ser compilado (maze.c);

A regra run compila (se necessário) e já executa o programa, enquanto a regra clean remove o executável gerado, facilitando a limpeza do diretório do projeto.

Por fim, a diretiva .PHONY: all run clean declara que esses três alvos não correspondem a arquivos reais, evitando conflitos caso existam, por acaso, arquivos com esses mesmos nomes no diretório do projeto.

4.2 Instruções de compilação

1. Certificar-se de que o compilador gcc está instalado no sistema;
2. Garantir que o arquivo-fonte maze.c e o Makefile estejam no mesmo diretório;
3. Posicionar o arquivo de labirinto (.txt) nesse mesmo diretório, ou anotar seu caminho completo;
4. No terminal, executar o comando "make" para gerar o executável maze;
5. Executar o programa com o comando "./maze"
6. Quando solicitado, informar o nome do arquivo do labirinto, incluindo a extensão .txt;
7. Caso deseje remover o executável gerado, utilizar o comando "make clean".